VocaVision

See clothing and food. Hear the description.

Project architecture and use case

1. What it is

VocaVision is an AI assistant that looks at a photo or a live camera view, recognises **clothing** and **food**, works out the **colour** of clothes, and then **describes the scene in a spoken sentence**, for example: *“I can see a white T-shirt.”* or *“I can see 3 hamburgers.”*

Use case

- **Accessibility:** a visually impaired person can point a camera at an outfit or a plate and hear what it is and what colour it is.
- **Everyday help:** choosing matching clothes, identifying a dish, or quick cataloguing of items.
- **Learning project:** shows how several AI models (object detection, classification, zero-shot vision-language) are combined into one product.

2. The big picture

The same idea runs in three forms. Every form follows one pipeline: **capture, detect, describe, show and speak**.

Form	Where it runs	Best for
Desktop app (Windows)	Double-click VocaVision. A local server starts and a window opens with the interface.	Most accurate: all models, including the CLIP cross-check
Android app (APK)	Entirely on the phone. No internet or PC needed.	Portable, live camera, fast
Original prototype	Python scripts with an OpenCV camera loop (main.py).	Development and testing

3. Processing pipeline

1. Capture
Photo upload, drag-and-drop, paste, or a live camera frame (laptop webcam, or a phone camera through Camo).

v

2. Detect (YOLOv8)
A custom-trained YOLOv8 model finds objects and draws boxes. It knows 20 classes: 10 foods and 10 clothing types. Each class has its own confidence bar (for example bag and sneaker need a higher score because they are often false alarms).

v

3. Cross-check (CLIP, desktop only)
CLIP, a pre-trained OpenAI vision-language model, compares the image with text such as “a photo of a pizza”. It catches cases where YOLO is overconfident and names clothing colour more reliably.

v

4. Colour
Pixels of the garment are grouped into colour clusters, converted to HSV and matched to names such as white, navy, beige. On desktop CLIP also votes on the colour.

v

5. Describe
Results are merged into one natural sentence. Repeats are counted (“3 hamburgers”) and joined with commas and “and”.

v

6. Show and speak
Annotated image, confidence, top matches and history appear on screen. The sentence is read aloud by the text-to-speech voice.

4. Desktop app architecture

Layer	Role
Interface (HTML / CSS / JavaScript)	The screen the user sees: upload and live-camera tabs, result card, history, light and dark theme, voice toggle, keyboard shortcuts. Opens in an Edge or Chrome window without browser bars, so it feels like a normal app.
Local server (Python, Flask)	Receives the picture from the interface, runs the AI, and returns the text and annotated image. Loads models once at start-up behind a loading splash, and closes itself when the window closes.
Vision engine	Coordinates YOLO, CLIP, colour detection and the fall-back classifiers, and builds the sentence.
Voice	The browser speech synthesis reads the sentence aloud, offline.

Everything runs locally on the computer, so photos never leave the machine.

5. Android app architecture

Part	Role
CameraX	Gets live camera frames and the preview, at 4:3 so boxes line up with the picture.
ONNX Runtime + YOLO	The same trained YOLOv8 model, exported to ONNX format and run on the phone CPU. Input size can be 320 (Fast), 416 (Balanced) or 512 (Accurate); a smaller size gives a higher frame rate.
Colour and sentence builder	Same colour ranges and sentence rules as the desktop app.
Android text-to-speech	Speaks a sentence only after it has stayed the same for about a second, so it does not babble on every frame.
Overlay and buttons	Boxes with labels, an FPS badge, Voice on/off, quality mode, flip camera, and a Photo button for gallery images.

Why it is fast: only the newest frame is processed (old ones are dropped), the model runs on a background thread so the preview never freezes, and memory buffers are reused instead of rebuilt every frame.

6. Models and data

Model	Type	Purpose
YOLOv8s (custom trained)	Object detection	Main detector for 20 food and clothing classes, trained on Open Images-derived data
CLIP ViT-B/32 (OpenAI, pre-trained)	Zero-shot image classification	Cross-check and colour; not trained by the team
Food classifier (TensorFlow)	Image classification	Fall-back for food when YOLO finds nothing
Clothing classifiers (TensorFlow, PyTorch)	Image classification	Fall-back for clothing
Colour detector (OpenCV)	Clustering + HSV rules	Names the garment colour

Foods: apple pie, fried rice, hamburger, hot dog, ice cream, pizza, ramen, samosa, sushi, waffles.
Clothing: T-shirt, shirt, pullover, coat, dress, trousers, sneaker, sandal, ankle boot, bag.

7. Strengths and limits

- Works offline and keeps photos private; the interface is simple and speaks results.
- Good, clear photos with the item centred give correct answers. The Android app trades some accuracy (no CLIP) for portability and speed.
- YOLO is weaker on T-shirts worn by a person because training data for that class was small. More data and retraining is the real fix.
- Only the 20 listed classes are recognised; anything else gives a polite “not sure” message.
- Testing so far is a small manual check, not a formal benchmark.

8. Tech stack and credits

Area	Tools
AI	YOLOv8 (Ultralytics, PyTorch), CLIP (Hugging Face Transformers), TensorFlow / Keras, OpenCV, ONNX Runtime

Area	Tools
Desktop	Python 3.11, Flask, HTML / CSS / JavaScript, Edge app window, Git LFS for large model files
Android	Kotlin, CameraX, ONNX Runtime Mobile, Android TextToSpeech, Gradle
Team	Original project by Raj (RAJ-A58/VocaVision) and group members. Desktop interface, CLIP cross-check, camera picker and the Android app were added on top.